

BRAC UNIVERSITY

Project Report

Bank Marketing Term Deposit Subscription Prediction

Predicting Client Subscription to a Term Deposit Using Supervised and Unsupervised Machine Learning Models

Prepared by

Name	Student ID
Deema Tabassum Khan	23101162
Farhan Shahriar	23101214

Course: CSE422 — Artificial Intelligence, Section: 05

Date of Submission: 5 September, 2026

Table of Contents

1. Introduction **3**

2. Dataset Description **3**

2.1 Dataset Overview 3

2.2 Feature Types 3

2.3 Do the Categorical Variables Need Encoding? 4

2.4 Correlation Analysis 4

2.5 Imbalanced Dataset 5

2.6 Exploratory Data Analysis (EDA) 6

3. Dataset Pre-processing **13**

3.1 Problem: Duplicate Rows 13

3.2 Problem: Null / Missing Values (Disguised as "Unknown") 13

3.3 Problem: Categorical (Non-Numeric) Values 14

3.4 Problem: Feature Scaling 14

3.5 Putting It Together: Pipeline and ColumnTransformer 15

4. Dataset Splitting **15**

5. Model Training & Testing **15**

5.1 K-Nearest Neighbors (KNN) 15

5.2 Decision Tree 17

5.3 Logistic Regression 17

5.4 Neural Network (MLPClassifier) 18

5.5 Unsupervised Learning: K-Means Clustering 19

6. Model Selection / Comparison Analysis **20**

7. Conclusion **22**

1. Introduction

Direct marketing campaigns conducted over the phone remain one of the most common ways banks promote financial products such as term deposits. However, contacting every client in a large customer base is expensive and often intrusive, and only a small fraction of contacted clients actually subscribe. This project aims to build machine learning models that can predict, before or during a campaign, whether a client is likely to subscribe to a term deposit based on their demographic profile, past interactions with the bank, and macroeconomic context at the time of contact.

The dataset used in this project is the well-known Bank Marketing dataset, which records the outcomes of telephone-based direct marketing campaigns carried out by a Portuguese banking institution. The target variable, y, indicates whether the client subscribed to a term deposit ("yes") or not ("no").

The motivation behind this project is twofold. First, from a business perspective, an accurate predictive model allows the bank to prioritize contacting clients who are more likely to say "yes", improving campaign efficiency and reducing costs associated with contacting uninterested clients. Second, from an academic perspective, this dataset is highly imbalanced and contains a mix of numerical and categorical features, which makes it an excellent case study for practicing real-world data pre-processing, handling class imbalance, and comparing multiple supervised classification algorithms (K-Nearest Neighbors, Decision Tree, Logistic Regression, and a Neural Network) as well as an unsupervised approach (K-Means clustering) on the same problem.

2. Dataset Description

2.1 Dataset Overview

The dataset used is the Bank Marketing dataset ("Bank Marketing.csv"), which contains the results of phone-call based direct marketing campaigns of a Portuguese bank.

- Number of data points (rows): 41,188
- Number of features: 20 input features + 1 target variable (y) = 21 columns in total
- Problem type: Classification — the target variable y is categorical with two classes, "yes" and "no", representing whether the client subscribed to a term deposit. Since the output is a discrete class label rather than a continuous quantity, this is a binary classification problem rather than a regression problem.

The dataset shape and column information obtained directly from the notebook confirm this:

Property	Value
Total rows	41,188
Total columns	21 (20 features + target)
Numerical features	10
Categorical features	10 (+ target y)
Missing values (raw, before cleaning)	0 (nulls); however "unknown" is used as a placeholder category

2.2 Feature Types

The 20 input features are a mix of quantitative (numerical) and qualitative (categorical) variables:

Numerical (Quantitative) Features — 10

- age — client's age in years

- duration — last contact duration, in seconds
- campaign — number of contacts performed during this campaign for this client
- pdays — number of days since the client was last contacted from a previous campaign (999 means not previously contacted)
- previous — number of contacts performed before this campaign for this client
- emp.var.rate — employment variation rate (quarterly economic indicator)
- cons.price.idx — consumer price index (monthly economic indicator)
- cons.conf.idx — consumer confidence index (monthly economic indicator)
- euribor3m — Euribor 3-month rate (daily economic indicator)
- nr.employed — number of employees (quarterly economic indicator)

Categorical (Qualitative) Features — 10

- job, marital, education, default, housing, loan — client demographic and financial attributes
- contact, month, day_of_week — attributes of how/when the client was contacted
- poutcome — outcome of the previous marketing campaign

The target variable y (subscribed: yes/no) is also categorical.

2.3 Do the Categorical Variables Need Encoding?

Yes. Machine learning algorithms such as KNN, Logistic Regression, and Neural Networks operate on numerical distance and weighted-sum computations, so text categories such as "married" or "admin." cannot be used directly — they must be converted into a numerical representation. In this project, one-hot encoding (via OneHotEncoder) was applied to all categorical columns so that each category becomes its own binary (0/1) indicator column, avoiding a false ordinal relationship between categories that have no natural order (e.g., job types). The numerical features, on the other hand, were standardized (scaled) rather than encoded, since they are already numeric but exist on very different scales (e.g., age in years vs. nr.employed in thousands).

2.4 Correlation Analysis

A correlation matrix was computed for all 10 numerical features and visualized as a heatmap:

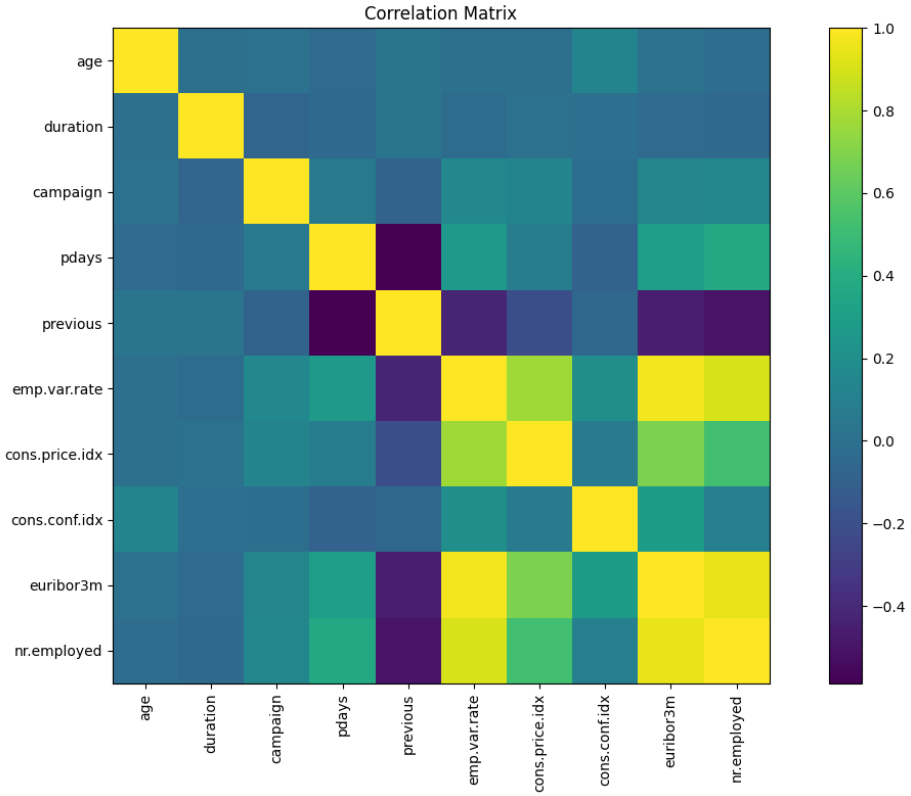


Figure 1: Correlation heatmap of numerical features.

From the correlation heatmap, several macroeconomic indicators are strongly correlated with each other: emp.var.rate, euribor3m, and nr.employed all show strong positive correlation (bright yellow-green cells), which makes intuitive sense since they all move together during periods of economic expansion or contraction. cons.price.idx is also moderately correlated with emp.var.rate and euribor3m. On the other hand, pdays and previous are strongly negatively correlated (dark purple), which is expected since a client who was contacted more times previously (higher previous) would naturally tend to have a smaller number of days since that last contact only when they were actually re-contacted, while clients who were never previously contacted are coded with pdays = 999. Features like duration and campaign are largely uncorrelated with the other numerical features, meaning they contribute relatively independent information. Overall, no pair of features is perfectly collinear (correlation ≈ 1), so multicollinearity is present only among the macroeconomic indicators and does not require dropping columns outright, though it is worth keeping in mind when interpreting model coefficients (e.g., in Logistic Regression).

2.5 Imbalanced Dataset

The target variable y is highly imbalanced. After removing duplicate rows, the class distribution is:

Class	Count	Percentage
no (did not subscribe)	36,537	88.73%
yes (subscribed)	4,639	11.27%

This means that a naive model which always predicts "no" would already achieve about 88.7% accuracy while being completely useless for identifying potential subscribers. This imbalance is clearly visible in the bar chart of the target class distribution:

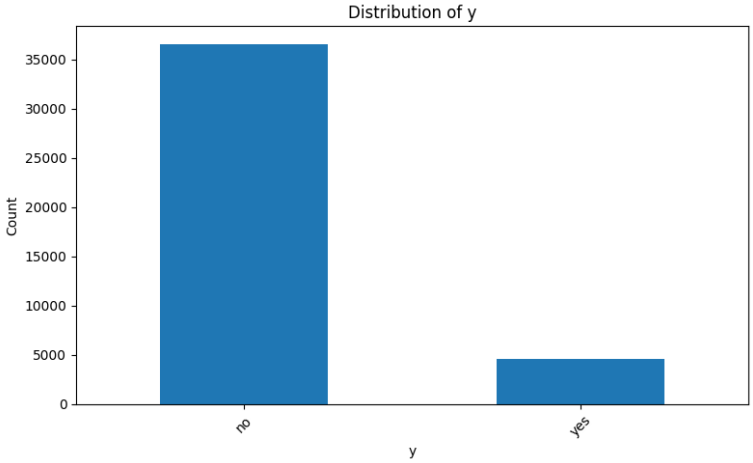


Figure 2: Class distribution of the target variable y, showing severe imbalance (~89% "no" vs. ~11% "yes").

Because of this imbalance, accuracy alone is not a reliable metric for this problem — precision, recall, F1-score, and AUC (which are much less sensitive to class imbalance) were also reported for every model, and class_weight="balanced" was used for the Decision Tree and Logistic Regression models to compensate for the skew during training.

2.6 Exploratory Data Analysis (EDA)

2.6.1 Distribution of Numerical Features

Histograms of all 10 numerical features were plotted to understand their shape and spread:

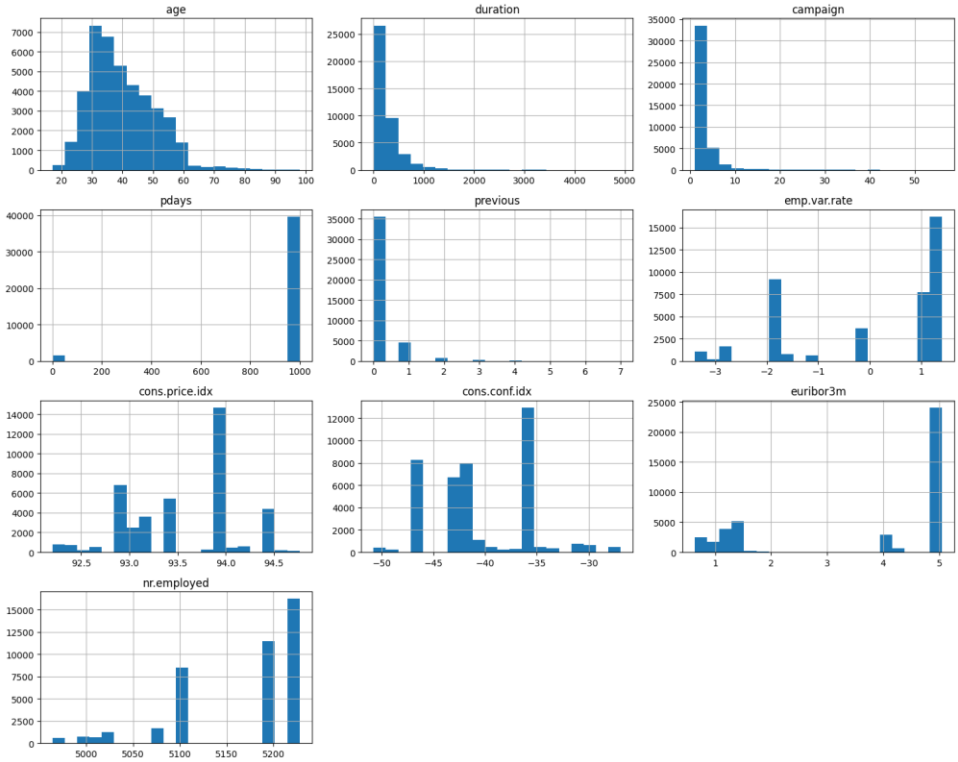


Figure 3: Histograms of all numerical features.

Several important patterns emerge. age is right-skewed and roughly bell-shaped, concentrated between 25 and 60 years. duration and campaign are heavily right-skewed with a long tail of large values, meaning most calls are short and most clients are contacted only a few times, but a handful of calls/contacts are extreme outliers. pdays is dominated almost entirely by the value 999 ("not previously

contacted"), which explains its unusual histogram shape. The macroeconomic indicators (emp.var.rate, cons.price.idx, cons.conf.idx, euribor3m, nr.employed) each take on a small number of discrete/repeated values, because they change only monthly or quarterly rather than per client, which produces the distinctly "blocky" histograms rather than smooth continuous distributions.

2.6.2 Outlier Detection via Boxplots

Boxplots were generated for every numerical feature to detect outliers:

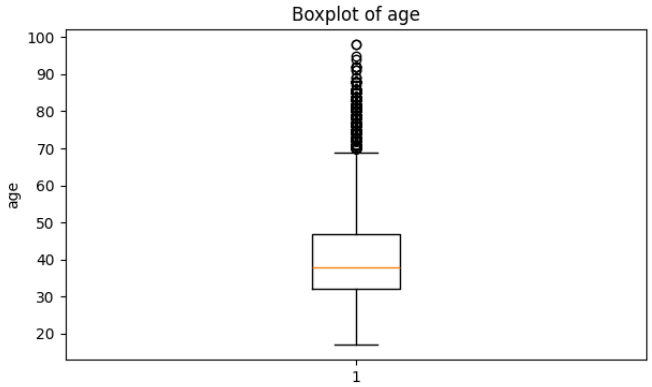


Figure 4a: Boxplot of age.

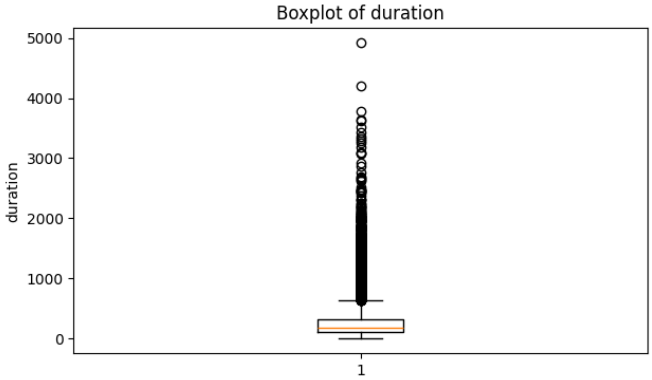


Figure 4b: Boxplot of duration — a large number of high-value outliers are visible.

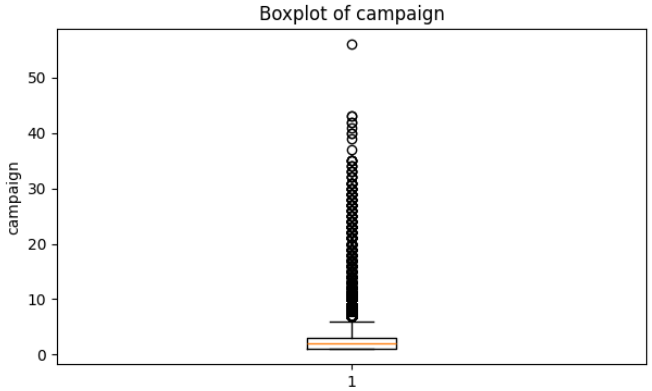


Figure 4c: Boxplot of campaign.

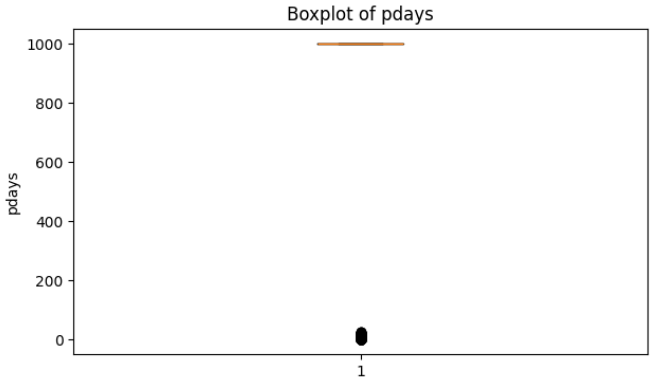


Figure 4d: Boxplot of pdays.

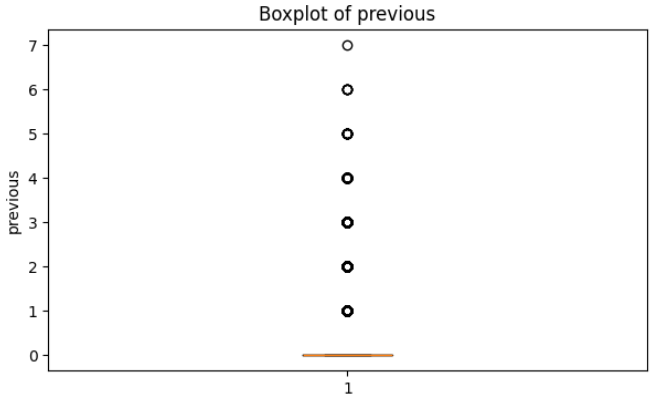


Figure 4e: Boxplot of previous.

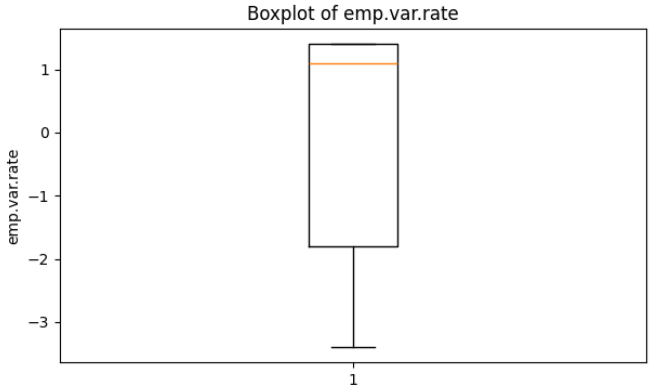


Figure 4f: Boxplot of emp.var.rate.

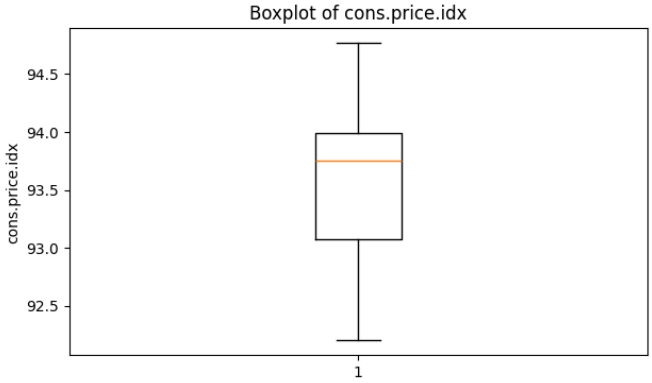


Figure 4g: Boxplot of cons.price.idx.

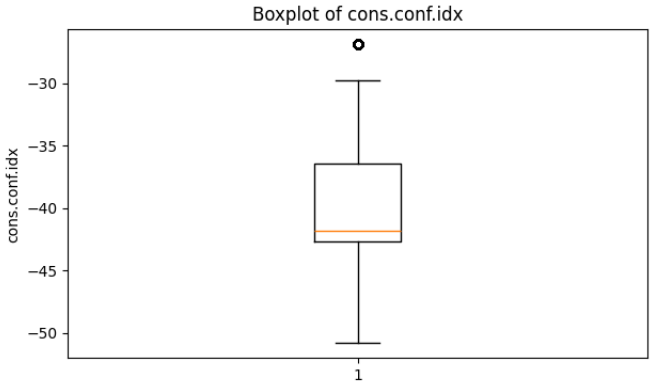


Figure 4h: Boxplot of cons.conf.idx.

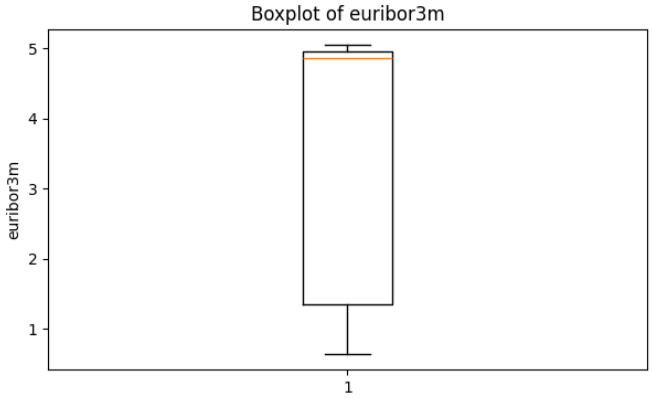


Figure 4i: Boxplot of euribor3m.



Figure 4j: Boxplot of nr.employed.

The most extreme outliers are found in duration and campaign, which both have a long tail of clients who either stayed on the call for an unusually long time or were contacted an unusually large number of times. age and previous show a moderate number of outliers on the higher end. These outliers were retained rather than removed, since (a) they are legitimate values rather than data-entry errors, and (b) tree-based and neural-network models can tolerate such outliers reasonably well, while the StandardScaler applied to Logistic Regression and KNN reduces (but does not eliminate) their influence.

2.6.3 Distribution of Categorical Features

Bar charts of every categorical feature (including the target) were plotted to inspect class balance within each feature:

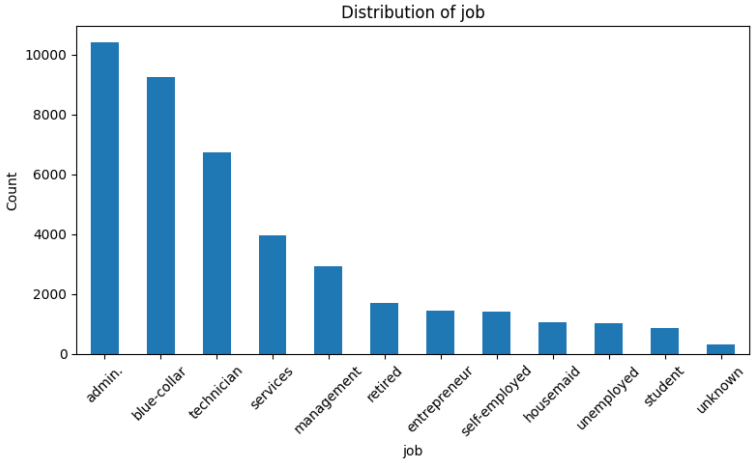


Figure 5a: Distribution of job.

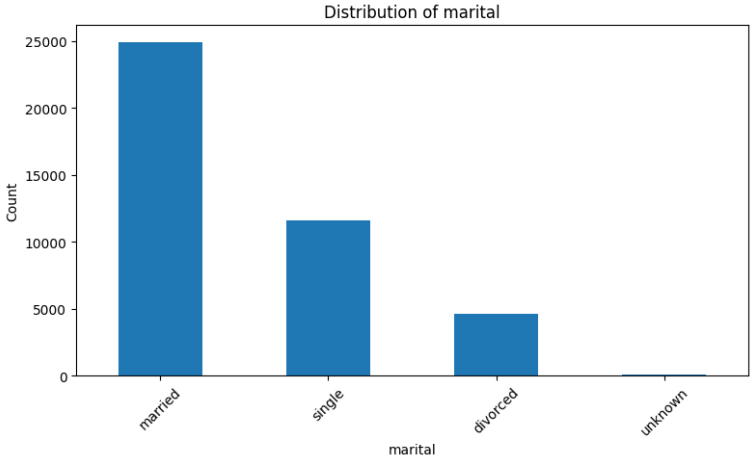


Figure 5b: Distribution of marital status.

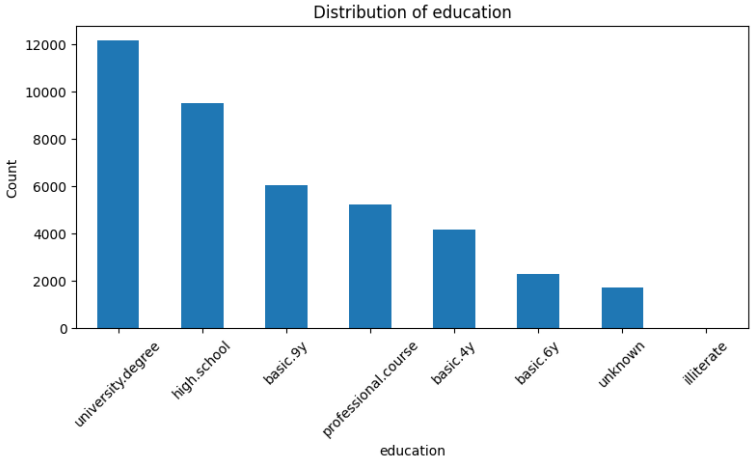


Figure 5c: Distribution of education.

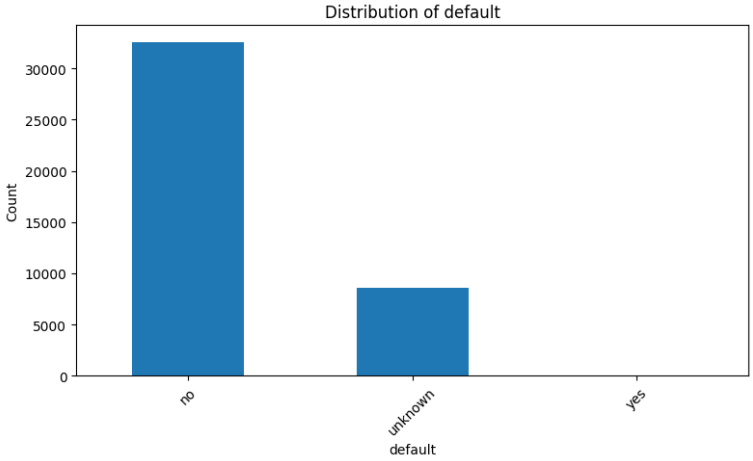


Figure 5d: Distribution of default (has credit in default).

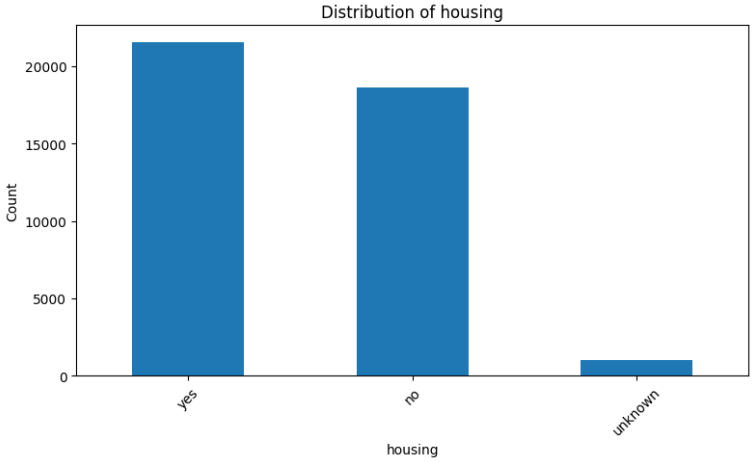


Figure 5e: Distribution of housing loan status.

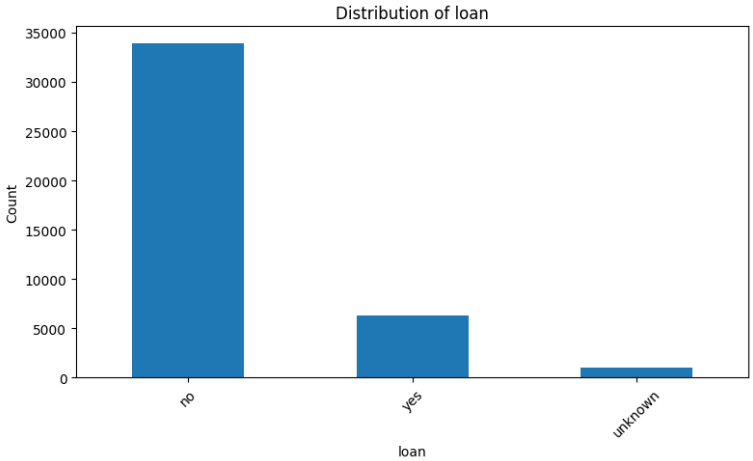


Figure 5f: Distribution of personal loan status.

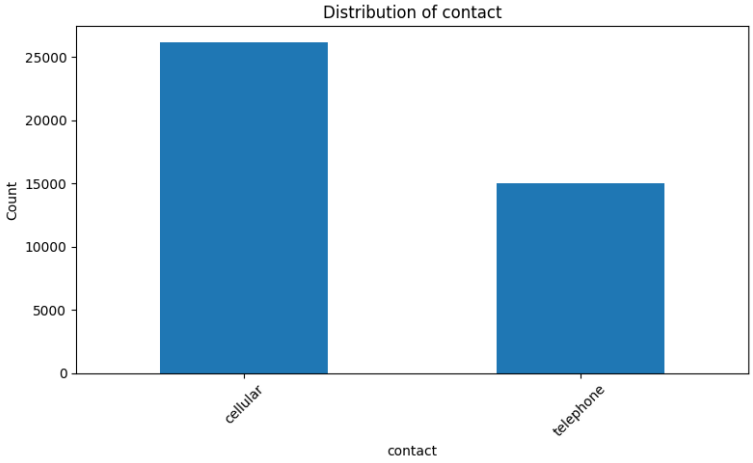


Figure 5g: Distribution of contact type.

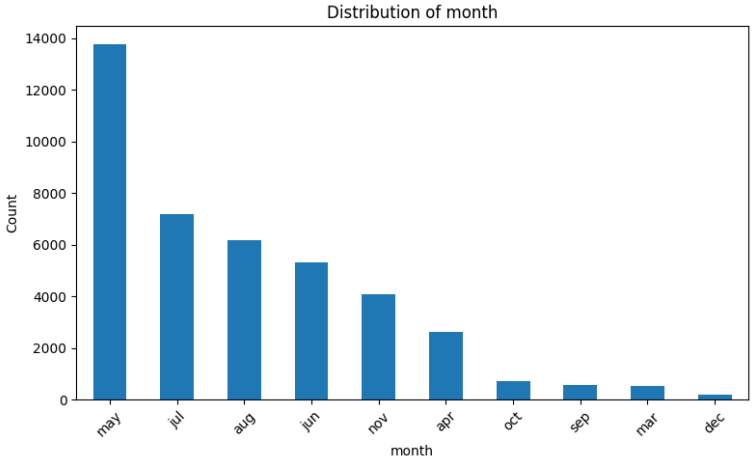


Figure 5h: Distribution of contact month.

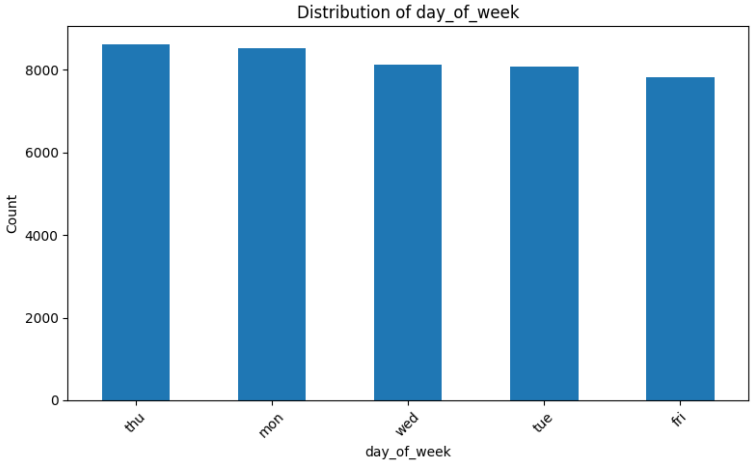


Figure 5i: Distribution of day of week.

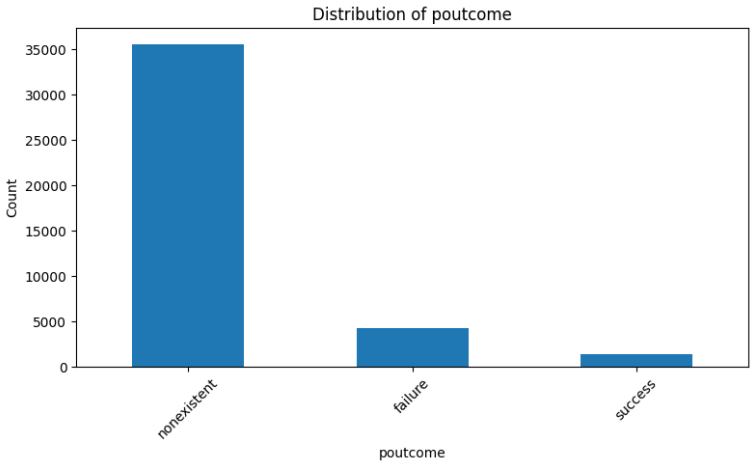


Figure 5j: Distribution of previous campaign outcome (poutcome).

Most clients work in administrative, blue-collar, or technician roles; are married; hold a university degree or high-school education; do not have credit in default; and were contacted via cellular phone rather than landline telephone. Campaign contacts are heavily concentrated in May, and for the vast majority of clients the outcome of any previous campaign is "nonexistent" (i.e., they were never part of an earlier campaign), which is consistent with the high count of pdays = 999 observed earlier. The default feature is particularly interesting: almost no clients are recorded as "yes" for default, and a large fraction (about 20.9%, discussed in the next section) are recorded as "unknown".

3. Dataset Pre-processing

As instructed, each problem identified in the raw dataset is discussed below immediately followed by the pre-processing technique applied to solve that specific problem, before moving on to the next problem.

3.1 Problem: Duplicate Rows

Before any other cleaning, the dataset was checked for fully duplicated rows using df.duplicated().sum(), which returned 12 duplicate rows sitting in the raw data.

Solution: Delete Duplicate Rows

Since a duplicate row carries no new information and would only bias the model by over-weighting whichever client record happened to be repeated, the 12 duplicate rows were simply deleted using df.drop_duplicates(). This reduced the dataset from 41,188 to 41,176 rows, and re-running df.duplicated().sum() afterward confirmed the count dropped to 0.

3.2 Problem: Null / Missing Values (Disguised as "Unknown")

A direct check with isnull().sum() shows zero true (NaN) missing values in the raw CSV. However, several categorical columns use the literal string "unknown" as a stand-in for missing information, which pandas does not recognize as a missing value by default — meaning a naive null-check would have wrongly concluded the data was complete. Explicitly counting these "unknown" placeholders reveals a substantial hidden missing-data problem:

Feature	"Unknown" Count	Percentage of Rows
default	8,596	20.88%
education	1,730	4.20%

Feature	"Unknown" Count	Percentage of Rows
housing	990	2.40%
loan	990	2.40%
job	330	0.80%
marital	80	0.19%

The default feature alone has almost 21% of its values recorded as "unknown", making it the most affected column.

Solution: Convert to NaN, then Impute (rather than Delete)

The cause behind choosing imputation over deletion is straightforward: default alone would cost roughly one-fifth of the entire dataset if its "unknown" rows were simply dropped, and the other affected columns would compound that loss further — deleting every affected row would discard a large amount of otherwise valid data and could bias the remaining sample. Instead, the "unknown" strings were first converted into proper, standard missing values using df.replace("unknown", np.nan), turning the disguised fault into an explicit one that scikit-learn's imputers can handle correctly. Missing values were then filled in as part of the preprocessing pipeline: numerical features (where applicable) use SimpleImputer(strategy="mean"), and categorical features use SimpleImputer(strategy="most_frequent") — replacing each missing category with the most common category in that column, a standard and safe strategy for nominal data. After this step, both the training and testing feature matrices report zero missing values, confirming the fix was applied successfully.

3.3 Problem: Categorical (Non-Numeric) Values

Ten of the twenty input features are categorical strings rather than numbers: job, marital, education, default, housing, loan, contact, month, day_of_week, and poutcome. Machine learning algorithms such as KNN, Logistic Regression, and Neural Networks rely on numerical distance and weighted-sum computations, so text categories such as "married" or "admin." cannot be fed into them directly — this is the cause that makes an explicit encoding step necessary.

Solution: One-Hot Encoding

Each categorical feature was converted using OneHotEncoder(handle_unknown="ignore"), which turns every category into its own binary (0/1) indicator column and safely ignores any category encountered at test time that was never seen during training. One-hot encoding (rather than simple label encoding, which assigns an arbitrary integer to each category) was chosen specifically because most of these categories have no natural order — for example, there is no numeric ranking implied between "admin.", "technician", and "blue-collar" job types — so label encoding would have wrongly introduced a false ordinal relationship that could mislead the model.

3.4 Problem: Feature Scaling

The numerical features are on very different scales — for example, age ranges roughly from 17 to 98, while nr.employed ranges from about 4,964 to 5,228, and duration can go into the thousands of seconds. This causes a problem for distance-based and gradient-based algorithms such as KNN, Logistic Regression, and the Neural Network, since a feature with naturally larger raw numeric values (like nr.employed) would otherwise dominate the distance or gradient calculations regardless of its actual predictive importance.

Solution: Standardization

All numerical input features were standardized using StandardScaler, which rescales each feature to have a mean of 0 and a standard deviation of 1. This puts every numerical feature on the same footing

so that no single feature dominates the model purely because of its raw scale, as per the requirement of the distance- and gradient-based models used later in this project.

3.5 Putting It Together: Pipeline and ColumnTransformer

The imputation, encoding, and scaling steps above were organized using scikit-learn Pipeline and ColumnTransformer objects, so that numerical and categorical columns are processed independently and consistently, and the same fitted transformations learned on the training set are re-applied to the test set (rather than being re-fit on the test set, which would cause data leakage).

After preprocessing, the feature matrix expands from 20 raw columns to 57 columns, since one-hot encoding replaces each categorical column with several binary indicator columns:

Dataset	Shape Before Split	Shape After Preprocessing
Full feature set (X)	(41,176, 20)	—
Training set	80% of X	(32,940, 57)
Testing set	20% of X	(8,236, 57)

4. Dataset Splitting

The cleaned dataset (41,176 rows after removing duplicates) was split into a training set and a testing set using an 80% / 20% split via train_test_split(test_size=0.2, random_state=42, stratify=y).

- Training set: 32,940 samples (80%)
- Testing set: 8,236 samples (20%)

The split was stratified on the target variable y. Stratification was chosen (rather than a plain random split) specifically because the dataset is heavily imbalanced (~88.7% "no" vs. ~11.3% "yes", as shown in Section 2.5). A plain random split risks producing a test set with a noticeably different class ratio than the training set purely by chance, which would make the reported evaluation metrics unreliable. Stratified splitting guarantees that both the training and testing sets preserve the same ~89:11 class ratio as the full dataset, giving a fair and representative evaluation of each model.

The fixed random_state=42 was used so that the split (and therefore all reported results) is reproducible.

5. Model Training & Testing

All models below were trained on the preprocessed training set (X_train_processed, y_train) and evaluated on the held-out preprocessed testing set (X_test_processed, y_test). y was label-encoded so that "yes" = 1 and "no" = 0, so precision/recall/F1/AUC are all reported with respect to correctly identifying clients who subscribe (the minority, and business-relevant, class).

5.1 K-Nearest Neighbors (KNN)

A KNeighborsClassifier was trained with n_neighbors = 5 (the default), using Euclidean distance over the standardized and one-hot-encoded feature space.

- Accuracy: 0.9027 (90.27%)
- Precision: 0.5924
- Recall: 0.4386
- F1 Score: 0.5040
- AUC: 0.8718

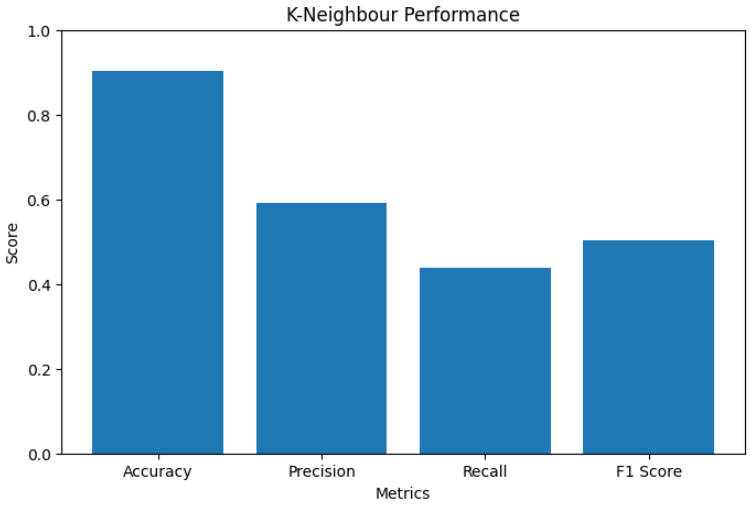


Figure 6: KNN performance metrics.

The confusion matrix for KNN on the test set is:

	Predicted: No	Predicted: Yes
Actual: No	7,028	280
Actual: Yes	521	407

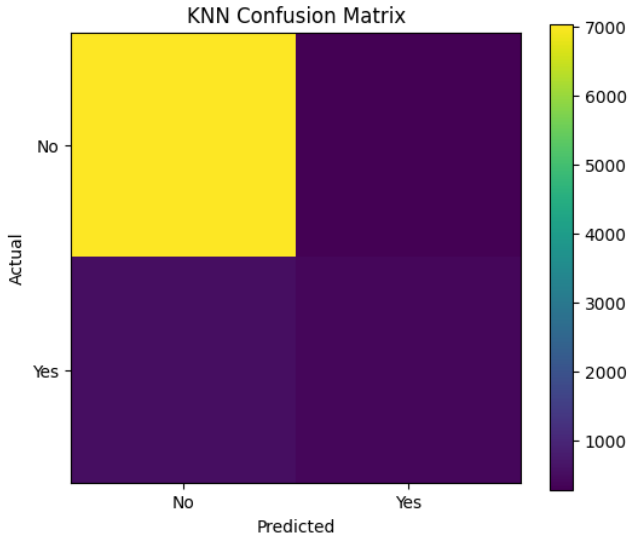


Figure 7: KNN confusion matrix (rows = actual class, columns = predicted class).

KNN achieves the highest raw accuracy among the four supervised models on the surface, but this is misleading given the class imbalance: its recall of only 0.44 means it misses more than half of the clients who actually subscribe (521 false negatives out of 928 true "yes" cases), which limits its usefulness for a marketing campaign whose goal is to identify likely subscribers.

5.2 Decision Tree

A DecisionTreeClassifier was trained with max_depth = 8, min_samples_leaf = 10, and class_weight = "balanced" (to counteract the class imbalance during training), with random_state = 42 for reproducibility.

- Accuracy: 0.8503 (85.03%)

- Precision: 0.4246
- Recall: 0.9256
- F1 Score: 0.5822
- AUC: 0.9341

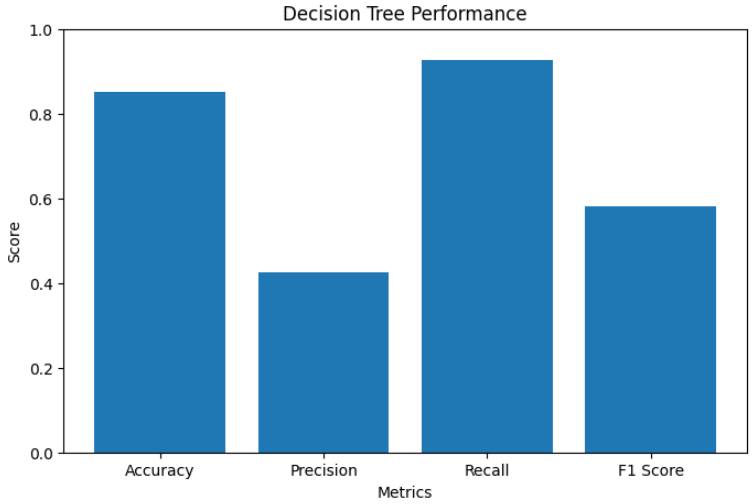


Figure 8: Decision Tree performance metrics.

The class_weight="balanced" setting clearly shifts the Decision Tree toward catching almost all true subscribers (recall = 0.93) at the cost of more false positives, which lowers precision and overall accuracy compared to KNN, but substantially raises the AUC to 0.93, indicating strong overall ranking ability between the two classes.

5.3 Logistic Regression

A LogisticRegression model was trained with class_weight = "balanced" and max_iter = 1000 to ensure convergence, with random_state = 42.

- Accuracy: 0.8628 (86.28%)
- Precision: 0.4462
- Recall: 0.9030
- F1 Score: 0.5973
- AUC: 0.9401

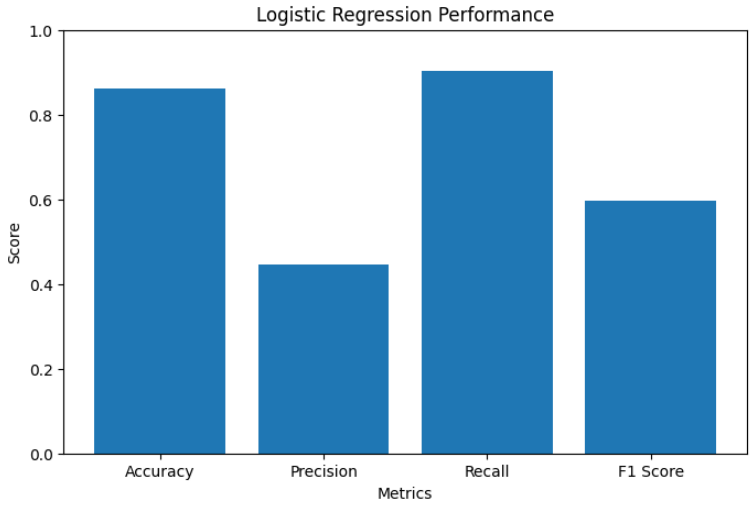


Figure 9: Logistic Regression performance metrics.

Logistic Regression achieves the second-highest AUC (0.94) among all models and a strong recall of 0.90, again a direct effect of class-balanced training. Its F1-score (0.597) is the best among all four supervised models, indicating the most favorable precision/recall trade-off overall.

5.4 Neural Network (MLPClassifier)

A feed-forward Neural Network was trained using scikit-learn's MLPClassifier with two hidden layers of size 64 and 32, ReLU activation, the Adam optimizer, early_stopping = True (to prevent overfitting), max_iter = 200, and random_state = 42.

- Accuracy: 0.9129 (91.29%)
- Precision: 0.6317
- Recall: 0.5453
- F1 Score: 0.5853
- AUC: 0.9458

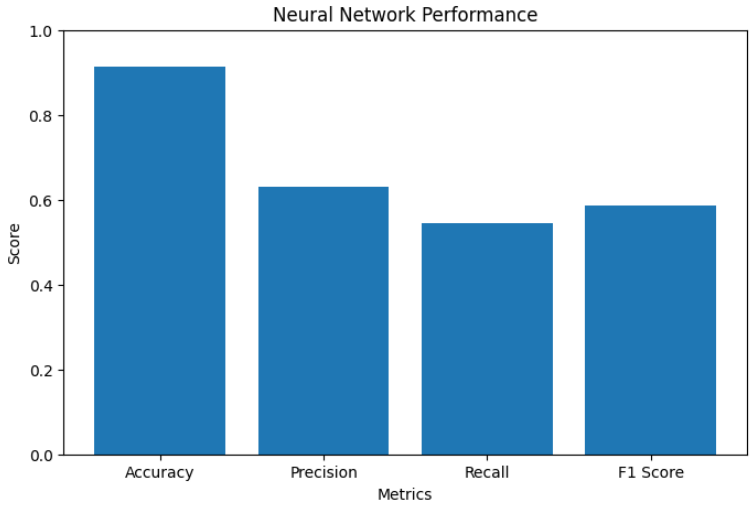


Figure 10: Neural Network performance metrics.

The Neural Network achieves the best accuracy (91.29%), the best precision (0.632), and the best AUC (0.946) of all five models trained, suggesting it captures non-linear interactions between features (e.g., between economic indicators and call duration) that the linear Logistic Regression model cannot. Its recall (0.545) is lower than the class-balanced Decision Tree and Logistic Regression models, since it was not explicitly trained with class weighting.

5.5 Unsupervised Learning: K-Means Clustering

As required, the same problem was also approached as an unsupervised learning task using K-Means clustering (ignoring the true labels during training). The optimal number of clusters was first investigated using the Elbow Method, plotting the within-cluster sum of squares (WCSS / inertia) for k = 2 to 10:

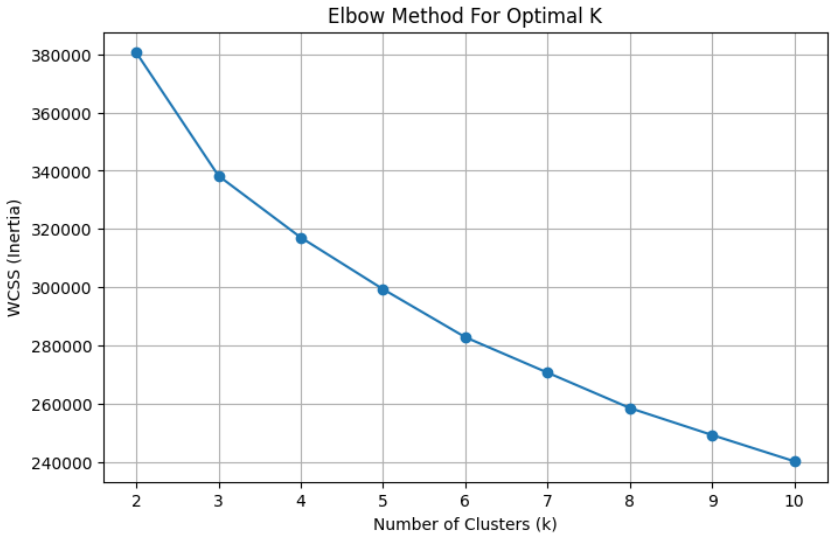


Figure 11: Elbow method for choosing the optimal number of clusters K.

The WCSS decreases steadily without an extremely sharp "elbow", which is common for high-dimensional, one-hot-encoded data; since the underlying business problem is fundamentally binary (subscribe vs. not subscribe), k = 2 was chosen to allow a direct comparison with the true binary labels.

KMeans(n_clusters=2, init="k-means++", n_init=10, random_state=42) was fit on the training data (without using y_train), and each test point was assigned to its nearest cluster centroid. Since K-Means clusters are unlabeled, the cluster whose majority label best matched "yes"/"no" was identified by comparing accuracy under both possible label assignments and keeping the higher-accuracy mapping.

- Accuracy: 0.7261 (72.61%)
- Precision: 0.2528
- Recall: 0.7317
- F1 Score: 0.3758
- AUC: 0.7285

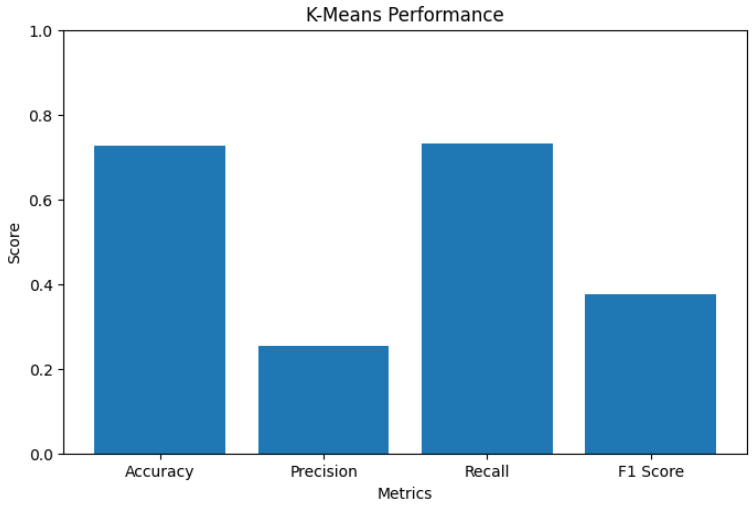


Figure 12: K-Means clustering performance metrics (evaluated against the true labels after best-mapping the clusters).

As expected, K-Means performs noticeably worse than every supervised model on accuracy and precision, since it groups clients purely by feature similarity in the transformed feature space without ever seeing the target label. Its recall (0.73) is reasonable — meaning the natural clusters do partially

align with subscription behavior — but its low precision (0.25) shows that the clusters also group together many non-subscribers with subscribers, confirming that unsupervised clustering alone is not competitive with supervised classification for this specific prediction task.

6. Model Selection / Comparison Analysis

The table below summarizes all five models' performance on the same held-out test set:

Model	Accuracy	Precision	Recall	F1 Score	AUC
KNN	0.9027	0.5924	0.4386	0.5040	0.8718
Decision Tree	0.8503	0.4246	0.9256	0.5822	0.9341
Logistic Regression	0.8628	0.4462	0.9030	0.5973	0.9401
Neural Network	0.9129	0.6317	0.5453	0.5853	0.9458
K-Means (unsupervised)	0.7261	0.2528	0.7317	0.3758	0.7285

A grouped bar chart comparing Accuracy, Precision, Recall, F1 Score, and AUC across the four supervised models makes the trade-offs easy to see side by side:

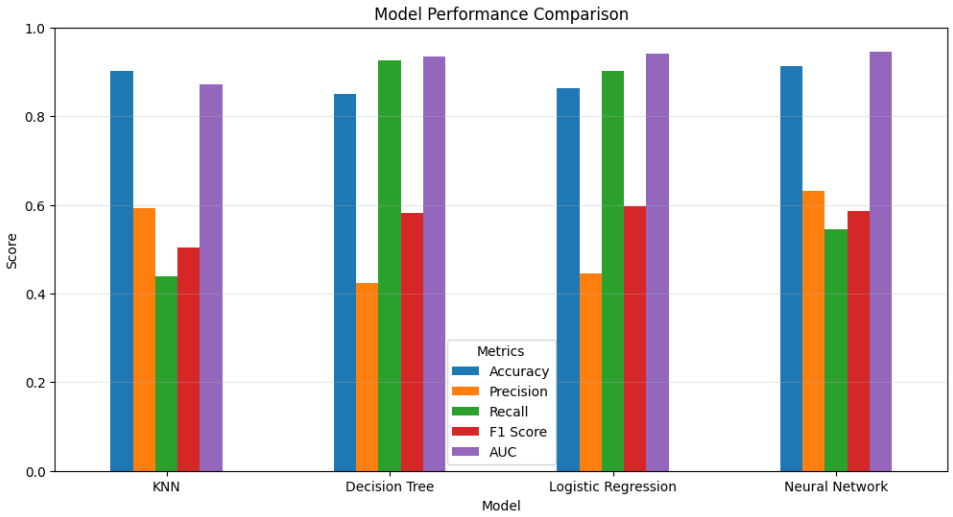


Figure 13: Side-by-side comparison of Accuracy, Precision, Recall, F1 Score, and AUC across the four supervised models.

6.1 Accuracy Comparison

The Neural Network has the highest accuracy (91.29%), narrowly ahead of KNN (90.27%). Logistic Regression and the Decision Tree trail behind (86.28% and 85.03% respectively) mainly because their class-balanced training deliberately sacrifices some overall accuracy in exchange for much better detection of the minority "yes" class. K-Means, being unsupervised, has the lowest accuracy at 72.61%.

6.2 Precision and Recall Comparison

There is a clear precision/recall trade-off across models. KNN and the Neural Network favor precision (0.59 and 0.63 respectively) but sacrifice recall (0.44 and 0.55), meaning that when they predict "yes" they are relatively more often correct, but they also miss a larger share of actual subscribers. The Decision Tree and Logistic Regression, both trained with class_weight="balanced", favor recall heavily (0.93 and 0.90 respectively) at the cost of precision (0.42 and 0.45), meaning they catch almost all real subscribers but also flag many non-subscribers as likely "yes". Which trade-off is preferable depends

on the bank's business priorities: if the cost of missing a real subscriber is high (e.g., a lost sale), the Decision Tree or Logistic Regression would be favored; if the cost of unnecessarily contacting a non-subscriber is the bigger concern, the Neural Network or KNN would be favored.

6.3 F1-Score Comparison

F1-score, the harmonic mean of precision and recall, provides a single balanced view. Logistic Regression achieves the highest F1-score (0.597), narrowly ahead of the Decision Tree (0.582) and the Neural Network (0.585), with KNN clearly behind (0.504) and K-Means far behind (0.376). This suggests Logistic Regression offers the best overall balance between correctly identifying subscribers and avoiding false alarms.

6.4 AUC Comparison

AUC (Area Under the ROC Curve) measures how well a model ranks positive examples above negative examples across all possible decision thresholds, independent of the specific 0.5 cutoff used elsewhere. On this metric, the Neural Network is the strongest model overall (AUC = 0.9458), closely followed by Logistic Regression (0.9401) and the Decision Tree (0.9341); KNN is noticeably weaker (0.8718), and K-Means, as expected for an unsupervised method, is the weakest (0.7285). Note that while AUC scores were computed for every supervised model using each model's predicted probabilities, explicit ROC curve plots were not generated in the accompanying notebook — only the scalar AUC values and the bar-chart comparison above are available from the executed code.

6.5 Overall Best Model

Taking accuracy, precision, F1-score, and AUC together, the Neural Network and Logistic Regression emerge as the two strongest supervised models overall — the Neural Network for raw predictive accuracy and ranking ability (AUC), and Logistic Regression for the best balance between precision and recall (F1-score) while remaining simple and interpretable. The Decision Tree is a close, more interpretable alternative to Logistic Regression with the highest raw recall of any model. KNN, while achieving good accuracy, underperforms on the metrics that matter most for this imbalanced problem (recall, F1). K-Means confirms that this task benefits substantially from using the labeled target during training, since the unsupervised approach falls well short of every supervised model.

7. Conclusion

This project applied a complete machine learning pipeline — exploratory data analysis, data cleaning, encoding, scaling, stratified train/test splitting, and multi-model training and evaluation — to the Bank Marketing dataset in order to predict whether a client will subscribe to a term deposit.

The results show that no single metric tells the whole story on an imbalanced dataset like this one. While KNN and the Neural Network report the highest raw accuracy, class-balanced models (Decision Tree and Logistic Regression) are considerably better at actually catching subscribers (higher recall), which is usually the more business-relevant goal in a marketing context — missing a genuine subscriber is typically more costly than making one extra phone call to a non-subscriber. The Neural Network stood out as the strongest model when considering AUC and precision together, suggesting it best captures the non-linear relationships between call duration, macroeconomic indicators, and client demographics that drive subscription decisions. The unsupervised K-Means baseline, as expected, performed clearly worse than every supervised model, confirming that the label information is genuinely useful and that this is not a problem where the natural clusters in the data already align well with subscription behavior.

We believe these results are explained by three main factors. First, the severe class imbalance (~89% vs. ~11%) makes accuracy alone a poor and even misleading metric, and explains why the class-balanced models trade accuracy for recall. Second, the dataset mixes highly informative behavioral

features (such as call duration and previous campaign outcome) with slow-moving macroeconomic indicators that change only monthly or quarterly, so tree-based and neural-network models that can exploit interactions between these different types of signals tend to outperform simpler models. Third, one-hot encoding a modest number of categorical features expanded the feature space from 20 to 57 dimensions, which favors models like the Neural Network and regularized Logistic Regression that handle higher-dimensional, sparse feature spaces gracefully, over distance-based methods like KNN whose performance tends to degrade as dimensionality grows (the curse of dimensionality).

A number of challenges were encountered while building this project. The most significant was handling the disguised missing values encoded as the string "unknown" rather than as standard nulls — these had to be explicitly detected and converted to NaN before they could be properly imputed. The severe class imbalance also required deliberate choices (stratified splitting and class_weight="balanced") to avoid building a model that trivially predicts the majority class. Finally, translating the unsupervised K-Means cluster assignments into a comparable evaluation against the true binary labels required an extra alignment step, since clustering does not inherently know which cluster corresponds to which real-world label.

As potential future work, techniques such as SMOTE (Synthetic Minority Over-sampling), hyperparameter tuning via grid or random search, and ensemble methods (e.g., Random Forest or Gradient Boosting) could be explored to further improve recall and F1-score for the minority "yes" class without sacrificing as much precision.